Benyamin Gidanian

Los Angeles, CA 90034 · (310) 613-3655 · bgidanian@gmail.com
GitHub · LinkedIn

Summary

Applied ML/AI engineer with hands-on experience building end-to-end systems in PyTorch and TensorFlow — from custom CNN architectures and domain adaptation pipelines to FastAPI inference services and Dockerized deployments. Experience spanning both research-grade modeling (astrophysical image classification, severe class imbalance, ROC/AUC benchmarking against published baselines) and production engineering (CI/CD, containerization, HuggingFace Hub integration). Strong foundations in training stability, transfer learning, and GPU-accelerated computation.

Technical Skills

Core ML/AI: PyTorch, TensorFlow, Keras, CUDA, Torchvision, BERT, Attention Mechanisms, Hugging Face Transformers, timm, ConvNeXt **Algorithms:** Supervised & Unsupervised Learning, CNNs, Neural Networks, Decision Trees, XGBoost, Recommender Systems, Reinforcement Learning (DQN)
ML Engineering: FastAPI, Docker, RESTful APIs, CI/CD (GitHub Actions), Transfer Learning, Hyperparameter Tuning, Model Evaluation (ROC/AUC, Precision, Recall, F1)
Data: NumPy, Pandas, Feature Engineering, Data Preprocessing
Languages: Python, SQL, JavaScript, C
Tools: Git, Linux, Bash, Pytest, Ruff, Pyright, uv, Jupyter Notebooks, HuggingFace Hub

---

Projects

PII Redaction API

*PyTorch · DeBERTa-v3 · HuggingFace Trainer · FastAPI · Docker · AWS ECS Fargate *(in progress)* GitHub

Fine-tuning `microsoft/deberta-v3-base` on the `ai4privacy/pii-masking-300k` dataset for named entity recognition across PII categories including names, emails, SSNs, and phone numbers. Implements BIO token classification with proper subword label alignment, evaluated via seqeval entity-level F1. Post-processing layer aggregates per-token logits into character-level spans and replaces them with typed placeholders (`[NAME]` , `[EMAIL]` , etc.) with a configurable confidence threshold. Deployed as a FastAPI service (API key auth, Pydantic validation, structured error responses) on AWS ECS Fargate via Docker + ECR, with a public Gradio demo on HuggingFace Spaces. Training tracked with Weights & Biases; fine-tuned model and card published to HuggingFace Hub.

Gravitational Lens Finding — ML4SCI DeepLense GSoC 2026 Evaluation

PyTorch · ConvNeXt-Tiny · EfficientNet-B0 · timm · HuggingFace Hub GitHub GitHub

- Achieved **0.9887 AUROC** on binary gravitational lens finding (16.6:1 class imbalance) using ConvNeXt-Tiny — within 0.006 of a purpose-built equivariant neural network (Parul et al., NeurIPS ML4PS 2024), despite no built-in rotational invariance.
- Achieved **0.98 macro AUROC** on 3-class lensing substructure classification (no substructure / subhalo / vortex) via two-phase fine-tuning of EfficientNet-B0 with discriminative learning rates.

- Addressed severe class imbalance through combined `WeightedRandomSampler` oversampling and calibrated `BCEWithLogitsLoss` pos-weighting, avoiding the AUC degradation that comes from using the raw imbalance ratio naively.
- Applied physics-motivated augmentation only (rotations ±180°, flips) — no color/intensity distortions that would corrupt physical flux measurements across telescope filter channels.

Flower Classification Inference Service

PyTorch · EfficientNet · FastAPI · Docker · HuggingFace Hub · GitHub Actions
GitHub · Model

- Achieved **>93% test accuracy** on the 102-class Oxford Flowers dataset via two-stage fine-tuning of EfficientNet-B0 — a 30-point improvement over a custom CNN (63%) trained from scratch on the same data.
- Implemented a two-stage transfer learning pipeline: classifier-only warm-up (20 epochs) followed by partial backbone unfreeze with **differential learning rates** per layer group (1e-5 → 1e-3), preserving low-level ImageNet features while adapting high-level representations.
- Applied class-balanced sample weighting via `sklearn.utils.compute_class_weight` to handle Oxford-102's uneven class distribution, combined with CosineAnnealingLR scheduling and AdamW optimization.
- Built a custom `FlowerDataset` class (parsing MATLAB `.mat` label files via `scipy.io`) and a `SubsetWithTransform` wrapper enabling per-split augmentation pipelines on shared dataset objects — filling a gap in native `torchvision` tooling.
- Served both models simultaneously via a FastAPI service with a shared lifespan context (`/predict`, `/predict/scratch`, `/health`), enforcing file-type, size (≤ 5 MB), and minimum dimension (224×224) validation.
- Integrated HuggingFace Hub for automatic weight download on cold start; containerized with Docker for reproducible deployment.
- Enforced code quality with a full CI pipeline (GitHub Actions): Ruff linting, Pyright type checking, Pytest suite (model-mocked for GPU-free CI), and Docker build verification on every push.

ML Fundamentals Library

Python · NumPy · Matplotlib · Pytest
GitHub

- Built a fully-tested, NumPy-only ML library from scratch spanning 6+ core algorithms including Linear/Logistic Regression, K-Means, Decision Trees, and a multi-layer Neural Network with manual backpropagation.
- Implemented gradient computation by hand, enabling transparent insight into forward/backward pass dynamics and training stability.
- Optimized numerical routines through vectorization for significant runtime performance gains over naive loop-based approaches.
- Enforced reproducibility and documentation quality with 100% Pytest coverage and a custom AST-based documentation generator.

Deep Q-Network (DQN) for Autonomous Lunar Landing

TensorFlow · Keras · Gymnasium · Python
GitHub

- Implemented a DQN agent from scratch to solve the LunarLander-v3 environment, reaching the solved threshold with a sustained rolling average score above 200.
- Engineered training stability via Experience Replay (deque buffer) and Target Networks with soft updates (τ=0.001) to reduce Q-value divergence.
- Accelerated training with `tf.function` graph-mode compilation, reducing per-episode compute time.

Crossword Puzzle Generator (Constraint Satisfaction Solver)

Python
GitHub

- Implemented a CSP solver to generate dense crossword grids, reducing the backtracking search space by over 80% using AC-3 arc consistency.
- Integrated MRV and LCV heuristics for improved constraint propagation speed and solution quality.

Professional Experience

One-on-One Talmud Tutor

Mesivta Bircas Yitzchok · Sept 2023 – March 2025

- Delivered individualized instruction over a 1.5-year engagement, developing structured curricula to accelerate comprehension and long-term retention.

Education & Certifications

Machine Learning Specialization — Stanford University & DeepLearning.AI (Coursera)
Supervised Learning, Neural Networks, Decision Trees, Unsupervised Learning, Recommender Systems

Harvard CS50 Suite — Harvard University (EdX)
CS50x (C, Python, SQL, Algorithms) · CS50p (OOP, Testing) · CS50ai (Search, Logic, Optimization) · CS50w (Django, React, CI/CD)